MIT 8103: ADVANCED DATABASE SYSTEMS

Name: Antonella Akosa | **Student ID:** 301929153 | **Case Study:** Library Management System - Unity Public Library Network (UPLN) |
GitHub Link: https://github.com/Antonellaakosa/advanced-database-portfolio

1. Introduction and Case Study

My project was a library system named "The Unity Public Library Network (UPLN)". Abuja is the library headquarters, with four branches in Lagos, Port Harcourt, Kano and Enugu. All branches have a common catalogue, and members can borrow and return books from any branch. The system keeps records of the branches, members, staff and books. It also keeps information on book categories, physical copies, authors and publishers. Members register, search for books, can borrow and return them, or reserve books. The library additionally carries out book transfers amongst branches, fines, payments and reports. All the data used is synthetic, and all money values are in Nigerian naira (NGN). I selected a library as it was familiar to me and appropriate for all five portfolios.

2. Portfolio Design, Implementation and Testing

2.1 Portfolio 1 (Database Design and Modelling)

Before designing the database, I studied the activities in the library and determined what information was required and how the information should be related. Next, I created the entity-relationship (ER) diagram and 17 tables in PostgreSQL 16. Book titles were separated from physical copies due to the fact that there may be multiple copies of the same book in the different branches. This minimised duplicate data and allowed for easier data tracking. I also wrote rules for the library that the system had to obey, such as a partial unique index that would not accept 2 active loans for the same copy, but still allow for a loan history. I added access roles, audit logging and password hashing and encryption to protect personal and payment information. These checks were made to ensure that the database accepted only valid records, controlled access, and that changes were properly documented to protect sensitive information. I also tried the payment rules. I found 2 errors initially: The audit trigger was recording the function owner (it should be the user making the change), and the SQL injection test was giving me a syntax error, not testing my permissions. I fixed both mistakes and repeated the tests.

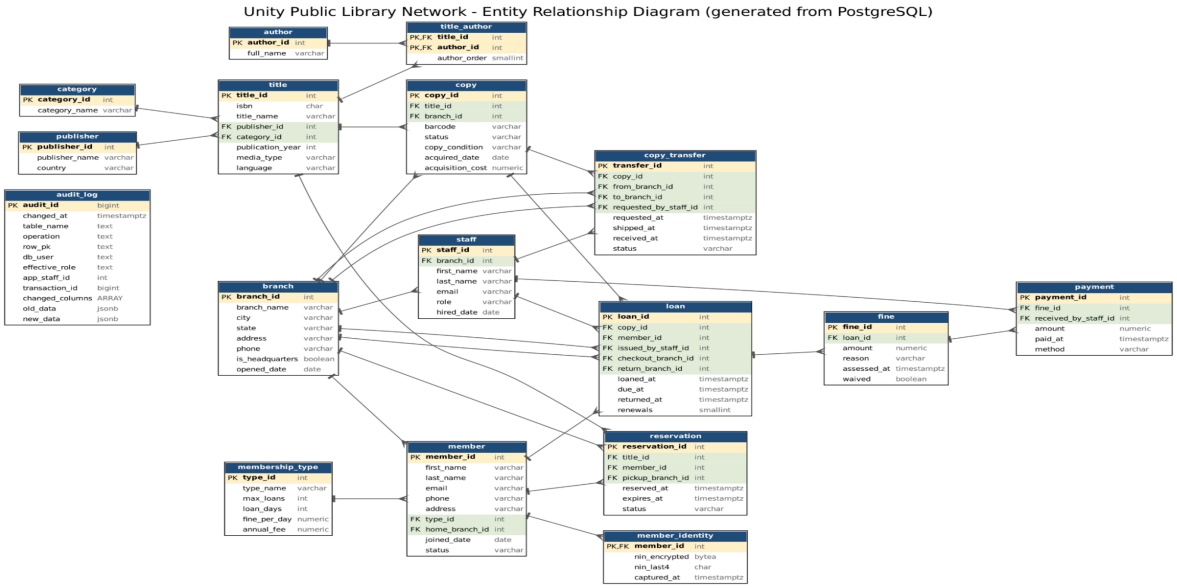
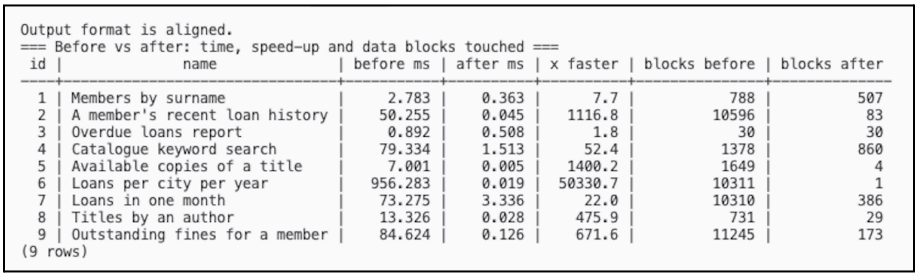


Figure 1. UPLN entity-relationship diagram according to the database design

2.2 Portfolio 2 (Query Processing and Optimisation)

In order to perform the tests properly, I created a separate (test) PostgreSQL database with 1,000,010 loan records, as the earlier set was not large enough to test the query. I ran 9 SQL queries related to the library operations, including the number of books in the library, the number of copies of a book available, outstanding fines and so on. I then compared their execution plans and results before and after adding indexes to see how it affected the performance. As I looked through the plans, I saw some queries that would have to pass through many records to return a few. The first one was the query that asks for a member's most recent 10 loans, which was done in a parallel sequential scan in the

original PostgreSQL. I created a composite index on member ID and loan date, which enabled it to use an index scan. The execution time then decreased from 50.255ms to 0.045ms. I also used a materialised view to make a report on the number of loans per city and year. It cut down the report time from 956.283 ms to 0.019 ms, but this did not include the time to refresh the view. For another test, I looked at how inserting records was impacted by adding additional indexes. With the additional indexes, PostgreSQL was able to insert 200,000 records in 2,151 ms, whereas it took 361 ms without them. This indicated that indexes can enhance the performance of queries but can also cause increased time for record insertion.

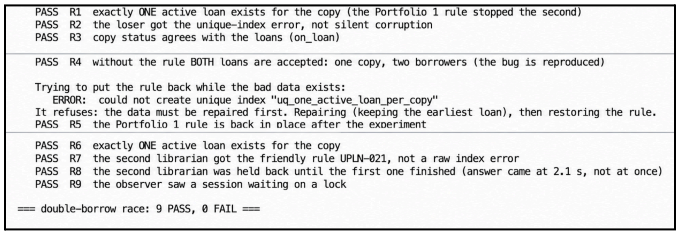


```
Output format is aligned.
=== Before vs after: time, speed-up and data blocks touched ===
 id |              name                | before ms | after ms | x faster | blocks before | blocks after
----+----------------------------------+-----------+----------+----------+---------------+--------------
  1 | Members by surname               |     2.783 |    0.363 |      7.7 |           788 |          507
  2 | A member's recent loan history   |    50.255 |    0.045 |   1116.8 |         10596 |           83
  3 | Overdue loans report             |     0.892 |    0.508 |      1.8 |            30 |           30
  4 | Catalogue keyword search         |    79.334 |    1.513 |     52.4 |          1378 |          860
  5 | Available copies of a title      |     7.001 |    0.005 |   1400.2 |          1649 |            4
  6 | Loans per city per year          |   956.283 |    0.019 |  50330.7 |         10311 |            1
  7 | Loans in one month               |    73.275 |    3.336 |     22.0 |         10310 |          386
  8 | Titles by an author              |    13.326 |    0.028 |    475.9 |           731 |           29
  9 | Outstanding fines for a member   |    84.624 |    0.126 |    671.6 |         11245 |          173
(9 rows)
```

Figure 2. Comparison of query execution times before and after optimisation

2.3 Portfolio 3 (Transactions and Concurrency Control)

I wanted to do some tests to determine if the database (PostgreSQL) would be able to handle transactions on the same records as other library staff would be accessing them at the same time. To illustrate possible conflicts, I set up a separate test database and performed transactions via various sessions. After I temporarily removed the partial unique index, in the double-borrow test, 2 librarians were able to borrow the same one. After I regained control of the constraint, PostgreSQL refused to accept the second loan. I also tried a checkout function that locks the copy before the second librarian attempted a checkout, which caused a refusal. The same issue with fine payments. The amount of NGN 1000 was outstanding; a total of NGN 800 was paid by each of the two cashiers, resulting in an overpayment of NGN 600 without locking the money. This was prevented by the payment guard. I also looked at transaction isolation, rollbacks, savepoints, lost updates, write skew and deadlocks. All 43 checks passed, confirming that the concurrent transactions were not the source of duplicate loans and overpayments.



```
  PASS  R1  exactly ONE active loan exists for the copy (the Portfolio 1 rule stopped the second)
  PASS  R2  the loser got the unique-index error, not silent corruption
  PASS  R3  copy status agrees with the loans (on_loan)

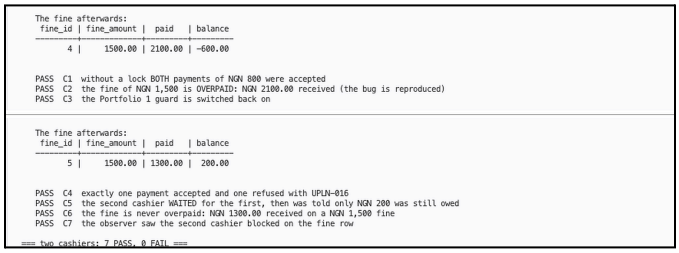
  PASS  R4  without the rule BOTH loans are accepted: one copy, two borrowers (the bug is reproduced)

  Trying to put the rule back while the bad data exists:
    ERROR:  could not create unique index "uq_one_active_loan_per_copy"
  It refuses: the data must be repaired first. Repairing (keeping the earliest loan), then restoring the rule.
  PASS  R5  the Portfolio 1 rule is back in place after the experiment

  PASS  R6  exactly ONE active loan exists for the copy
  PASS  R7  the second librarian got the friendly rule UPLN-021, not a raw index error
  PASS  R8  the second librarian was held back until the first one finished (answer came at 2.1 s, not at once)
  PASS  R9  the observer saw a session waiting on a lock

=== double-borrow race: 9 PASS, 0 FAIL ===
```

Figure 3a. Double-borrow test



```
The fine afterwards:
 fine_id | fine_amount |  paid   | balance
---------+-------------+---------+---------
       4 |     1500.00 | 2100.00 | -600.00


PASS  C1  without a lock BOTH payments of NGN 800 were accepted
PASS  C2  the fine of NGN 1,500 is OVERPAID: NGN 2100.00 received (the bug is reproduced)
PASS  C3  the Portfolio 1 guard is switched back on

The fine afterwards:
 fine_id | fine_amount |  paid   | balance
---------+-------------+---------+---------
       5 |     1500.00 | 1300.00 |  200.00

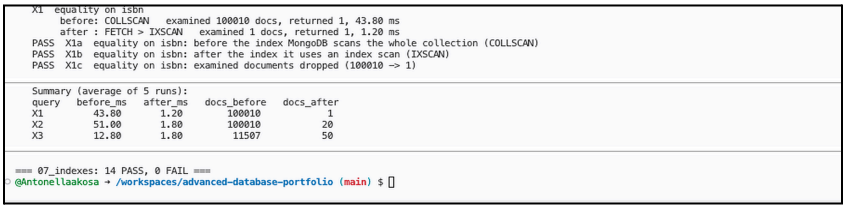

PASS  C4  exactly one payment accepted and one refused with UPLN-016
PASS  C5  the second cashier WAITED for the first, then was told only NGN 200 was still owed
PASS  C6  the fine is never overpaid: NGN 1300.00 received on a NGN 1,500 fine
PASS  C7  the observer saw the second cashier blocked on the fine row

=== two cashiers: 7 PASS, 0 FAIL ===
```

Figure 3b. Cashier tests

2.4 Portfolio 4 (NoSQL and Advanced Data Models)

My goal was to identify different models in NoSQL options for the library and determine which models were suitable for the library operations. I modelled the catalogue using MongoDB 7, as the details of a book, authors, categories and branches are typically retrieved together. The initial move was 10 titles from PostgreSQL, and thereafter 100,000 title generation that included validation and indexing of the document's schema, along with 50,000 reviews to test document operations, validation and indexing of the document's schema. The average execution time was decreased from 43.80 ms to 1.20 ms, and an ISBN lookup on one document was examined instead of 100,010. They also showed sessions that were created with key values, branch counters with wide columns, and graph relationships for book recommendations inside of MongoDB. The various NoSQL models were shown as examples of how they may be useful to the library's needs, leaving loans, fines and payments in PostgreSQL because of the relational integrity and transaction requirements they need.



```
  X1  equality on isbn
      before: COLLSCAN   examined 100010 docs, returned 1, 43.80 ms
      after : FETCH > IXSCAN   examined 1 docs, returned 1, 1.20 ms
  PASS  X1a  equality on isbn: before the index MongoDB scans the whole collection (COLLSCAN)
  PASS  X1b  equality on isbn: after the index it uses an index scan (IXSCAN)
  PASS  X1c  equality on isbn: examined documents dropped (100010 -> 1)

  Summary (average of 5 runs):
  query   before_ms   after_ms   docs_before   docs_after
  X1         43.80       1.20        100010            1
  X2         51.00       1.80        100010           20
  X3         12.80       1.80         11507           50

=== 07_indexes: 14 PASS, 0 FAIL ===
@Antonellaakosa → /workspaces/advanced-database-portfolio (main) $
```

Figure 4. MongoDB index plan and benchmark results

2.5 Portfolio 5 (Distributed and Cloud Database Concepts)

With UPLN operations spread across several branches, I set up a four-instance PostgreSQL cluster on a single machine in one of the Codespaces I had to explore the issue of database availability and consistency. This comprised a primary/coordinator, a replica and two shards that were used to distribute loan records by branch. The coordinator was able to get 50,000 loans from both shards but had not been able to get back a report of the entire network, because one shard was not available to return incomplete loans. The following was also discovered: If the replica was unavailable during a replication test, synchronous commits could be backed up. The manual failover promoted the replica in 634 ms. Both times, the former primary had a split-brain issue, with the writes being accepted separately. I also performed a test of backup and point-in-time recovery to restore deleted records. Both replication, sharding and recovery were demonstrated in practice, even though it was still a single-machine lab environment and not a cloud deployment, and transactions across shards were not atomic.

```
----- Promote the replica -----
   the replica accepted its first write 634 ms after the crash (script-measured recovery time)
   PASS  F3  the promoted replica is writable (no longer in recovery)
   PASS  F4  recovery time was under 30 seconds (634 ms, with a human-free, scripted promotion)
   rows committed before the crash: 200, found on the new primary: 200  (lost: 0)
   PASS  F5  no committed row was lost (RPO observed = 0 rows; asynchronous replication does not guarantee this)

----- Is the rest of the system still working through the new primary? -----
   cross-shard loan count through the new primary: 50000
   PASS  F6  the new primary still reaches both shards (50,000 loans)
   PASS  F7  the reporting account works against the new primary

----- Danger: split brain -----
   the old primary restarted and accepts writes (in_recovery=f). A row written there: on old=1, on new=0
   PASS  F8  split brain is possible: the restarted old primary happily accepts writes that the new primary never sees
   PASS  F9  the old primary is fenced off (stopped); only the new primary takes writes

   Failback (rebuilding the old primary as a replica) is documented in the README, not scripted.

=== 07_failover: 9 PASS, 0 FAIL ===
```

Figure 5. Some of the output from the Replica promotion and split-brain test.

3. Technical Reflection

Prior to this project, I only had experience with MySQL databases. PostgreSQL and MongoDB were the first to introduce me to things I had seen more in theory. A challenge was to see if my tests generated the anticipated results for the appropriate reasons. In fact, one of the audit triggers was not set to the function owner, but to the user who made the change. This separate security test failed due to a syntax error, not a restriction on permission. Both required correction. I had to give the same consideration to my query execution plans and concurrency tests, in which the results did not always provide an explanation of what happened. The other problem that I had to address was that the PostgreSQL instances were unable to communicate with each other in this distributed database configuration. The failover experiment later showed that the former primary can take writes even after the promotion of the replica. The failover itself was successful, but the database was still susceptible to conflicting writes, which was especially important.

4. AI Use Declaration

I used Claude to design the script, explain the logic, and generate test ideas. I also used ChatGPT to create the portfolio summaries from the work I completed. I tested the scenarios, checked the screenshots and verified the reported results with my implementation to find any mistakes or inconsistencies. I had to review and correct the issues I observed while testing.

5. References

i. PostgreSQL Global Development Group. PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/

ii. MongoDB. MongoDB Manual, Version 7.0. https://www.mongodb.com/docs/v7.0/

iii. MIVA Open University. MIT 8103 Advanced Database Systems course materials.

iv. Brewer, E. (2012). CAP twelve years later: How the 'rules' have changed. Computer, 45(2), 23–29.